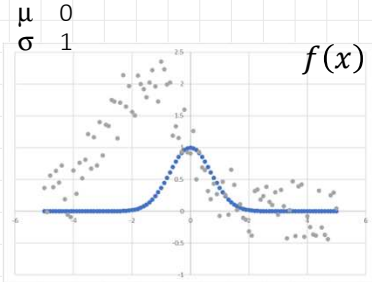
Parameter fitting

Parameter estimation

We have a series of data point and a parametric mathematical function.

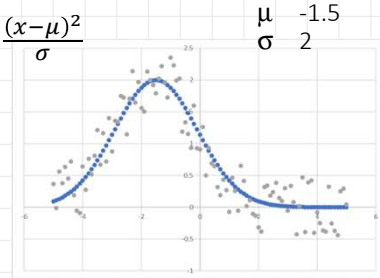
We want to find the parameter values that maximize the fit of the function to the points.

A 1
μ 0
σ 1



$$f(x) = Ae^{-\frac{(x-\mu)^2}{\sigma}}$$

A 2
μ -1.5
σ 2



Much theory

The topic, given the relevance, has been studied (largely independently) in several disciplines: mathematics, statistics, computer science, …

It goes under different names: parameter estimation, parameter fitting, curve fitting, (regression analysis).

There exists *Estimation theory*: "a branch of statistics that deals with estimating the values of parameters based on measured empirical data that has a random component. The parameters describe an underlying physical setting in such a way that their value affects the distribution of the measured data. An estimator attempts to approximate the unknown parameters using the measurements."

(*wikipedia*)

We will fly low, only scratching some theory

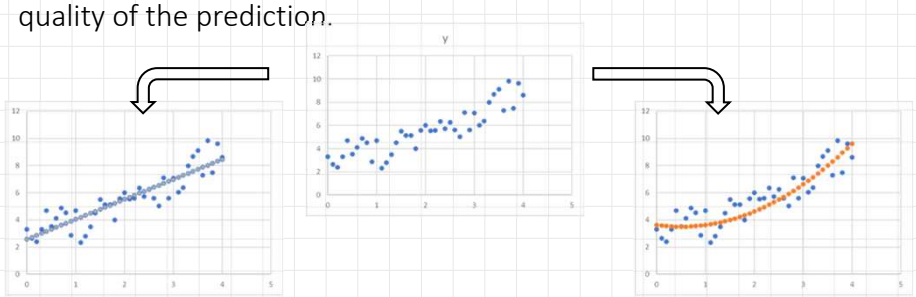
Optimization

As it is often the case, at the core there is optimization.

We want to minimize the difference between the sample points and those predicted by our model (base, mapping, baseline, …) function.

The model function is given. We want to minimize its prediction errors.

That is: we want minimize an error (loss, cost, …) function whose domain is the parameters space and whose image quantifies the quality of the prediction.



Simple error functions

$X = (x_1, \dots, x_n)$, sample data, $F = (f_1, \dots, f_n)$ model data.

Error functions include and are not limited to:

- BIAS: error arithmetic average $BIAS = \frac{\sum_{i=1}^{n}(x_i - f_i)}{n}$
- MAD (Mean Absolute Deviation): $MAD = \frac{\sum_{i=1}^{n}|x_i - f_i|}{n}$
- MSE (Mean Squared Error): $MSE = \frac{\sum_{i=1}^{n}(x_i - f_i)^2}{n}$
- Standard Error: $\sqrt{MSE}$
- MAPE (Mean Absolute Percent Error): $MAPE = \frac{\sum_{i=1}^{n}\frac{|x_i - f_i|}{f_i} * 100}{n}$

5

Python, error functions

```
import os, numpy as np, pandas as pd, matplotlib.pyplot as plt
# from statsmodels.tsa.stattools import acf
# Accuracy metrics
def forecast_accuracy(forecast, actual):
    mape = np.mean(np.abs(forecast - actual)/np.abs(actual))  # MAPE
    me   = np.mean(forecast - actual)           # ME
    mae  = np.mean(np.abs(forecast - actual))   # MAE
    mpe  = np.mean((forecast - actual)/actual)  # MPE
    rmse = np.mean((forecast - actual)**2)**.5  # RMSE
    corr = np.corrcoef(forecast, actual)[0,1]   # corr
    mins = np.amin(np.hstack([forecast[:,None], actual[:,None]]), axis=1)
    maxs = np.amax(np.hstack([forecast[:,None], actual[:,None]]), axis=1)
    minmax = 1 - np.mean(mins/maxs)             # minmax
    # acf1 = acf(forecast-actual)[1]            # ACF1
    return({'mape':mape, 'me':me, 'mae': mae, 'mpe': mpe,
            'rmse':rmse, 'corr':corr, 'minmax':minmax})

if __name__ == '__main__':
   os.chdir(os.path.dirname(os.path.abspath(__file__)))
   df = pd.read_csv('whichever.csv') # can try with testLinear.csv
   print( forecast_accuracy(forecast_values, actual.values) )
```


6

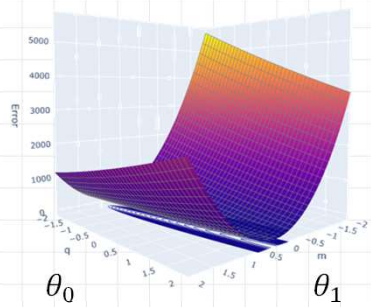
A simple case: linear regression

We have a set of datapoints in $\Re^n$ (on this slide in $\Re^2$),

We assume a linear model: $y = \theta_0 + \theta_1 x_1 + \ \dots + \theta_n x_n$

We choose as error function MSE (actually no mean, just the sum of the square errors)

error



The error surface to minimize

7

Linear regression, least squares

Example: linear regression where we estimate the coefficients of the equation $y = \theta_0 + \theta_1 x_1 + \ \dots + \theta_n x_n$

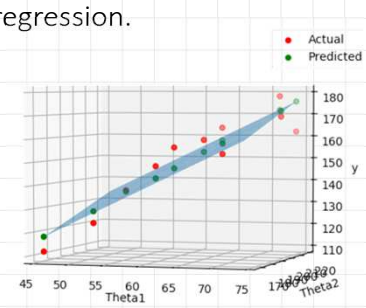
The optimal set of coefficients is determined by minimizing loss (cost, objective) function $\|\boldsymbol{y} - \boldsymbol{\Theta}'\boldsymbol{X}\|^2$, which is the maximum likelihood estimator for the task of linear regression.

Simple example: 1D array of n points.

We look for the interpolation line

$y = \theta_0 + \theta_1 x$ that minimizes the

mean square error on all data pairs (x_i, y_i):



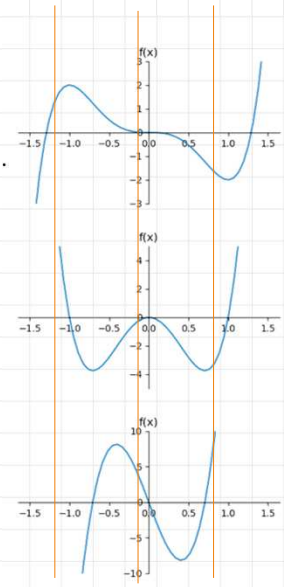
$$e(\boldsymbol{\Theta}) = \sum \left((x_i, y_i) - (x_i, \theta_0 + \theta_1 x_i)\right)^2$$

8

Some math we need

Let $f(x)$ be a real-valued, twice differentiable function. The value of x for which the first derivative $f'(x)$ is 0 corresponds to a maximum, a minimum or possibly a flex of $f(x)$.

- For a maximum the second derivative $f''(x)$ is negative.
- For a minimum the second derivative $f''(x)$ is positive.
- The second derivative is 0 for an inflexion point.



9

Some math we need

Jacobian

The Jacobian operator is a generalization of the derivative operator to vector-valued functions. The Jacobian matrix of a vector-valued function f is the matrix of all its first-order partial derivatives.

The Jacobian matrix represents the differential of f at every point where f is differentiable.

$$\mathrm{J} = \left[\frac{\partial f}{\partial x_1} \cdots \frac{\partial f}{\partial x_n}\right]$$

Throughout these slides, first order derivatives of $f(\mathrm{x})$ will be denoted either as $f'(\mathrm{x})$, or as $\mathrm{J}(\mathrm{x})$, or as $\nabla(\mathrm{x})$, according to my whims.

10

Some math we need

Hessian

The Hessian matrix is a square matrix of second-order partial derivatives of a function f.

It describes the local curvature of a function of many variables.

$$\mathbf{H} = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1\,\partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1\,\partial x_n} \\ \frac{\partial^2 f}{\partial x_2\,\partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2\,\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n\,\partial x_1} & \frac{\partial^2 f}{\partial x_n\,\partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}.$$

The trace of the Hessian matrix is known as the Laplacian operator

Throughout these slides, second order derivatives of $f(\mathrm{x})$ will be denoted either as $f''(\mathrm{x})$, or as $\mathrm{H}(\mathrm{x})$, or as $\nabla^2(\mathrm{x})$, according to my whims.

Least square, analytical

Analytically[1], having the average values $\bar{x}$ and $\bar{y}$: coefficients

$$\theta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} \qquad \left(= \frac{n \cdot \sum x_i y_i - \sum x_i \cdot \sum y_i}{n \cdot \sum x_i^2 - (\sum x_i)^2} \right)$$

and:

$$\theta_0 = \bar{y} - \theta_1 \cdot \bar{x} \qquad \left(= \frac{\sum y_i \cdot \sum x_i^2 - \sum x_i \cdot \sum x_i y_i}{n \cdot \sum x_i^2 - (\sum x_i)^2} \right)$$

Python:

```
th1 = ((X*y).mean() - X.mean()*y.mean())/((X**2).mean()-(X.mean())**2)
th0 = y.mean() - th1*X.mean()
```

If the number of elements in X increases, so does the load on CPU/GPU, maybe too much. We can ***approximate*** the solution by minimizing mean squared error by gradient descent (GD).

[1] You can derive this by yourself or read the derivation f.e. in https://math.stackexchange.com/questions/131590/derivation-of-the-formula-for-ordinary-least-squares-linear-regression

Gradient descent

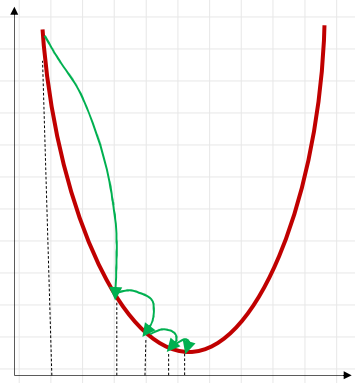
Gradient (or steepest) descent (Cauchy,1847) is the core of many learning algorithms (machine learning, deep learning, …).

GD is a first-order iterative algorithm for finding the local minimum of a differentiable function $f(x)$ (it's *gradient ascent* for the maximum).

To find a local minimum it starts from a point x_0 and takes steps proportional to the negative of the gradient (or of an approximate gradient) of the function at the current point.

The length of the steps is dictated by a parameter, the learning rate (λ).

$$x_{t+1} = x_t - \lambda \nabla f(x_t)$$



Gradient descent, example

Cost function $f(x) = x^3 - 4x^2 + 6$

Derivative $f'(x) = 3x^2 - 8x$ (the slope of $f(x)$ at any given x)

Start with any value of x, say 0.5 and $\lambda = 0.05$

Iterate a number of times, always re-calculating the value of x.

$x = x - \lambda * f'(x)$

$x = 0.5 - (-3.25*0.05) = 0.6625$

Use $x = 0.6625$ in second iteration

$x = 0.6625 + (3.983*0.05) = 0.86165$

and so on for a number of iterations, or until a predefined value of precision is reached.

Step size

Choice of λ matters a lot in practice:

$$\underset{x}{\text{minimize}}\ 2x_1^2 + x_2^2 + x_1x_2 - 6x_1 - 5x_2$$

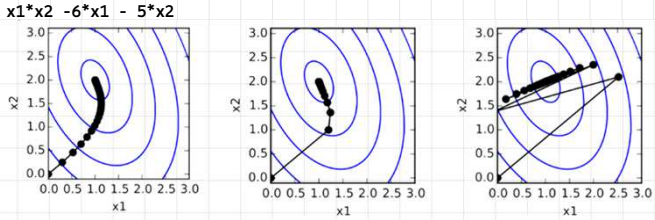
$$J = [4x_1 + x_2 - 6, 2x_2 + x_1 - 5]$$

```
def f(x1,x2):
   return 2*x1*x1 + x2*x2 + x1*x2 -6*x1 - 5*x2

def dfdx1(x1,x2):
   return 4*x1 + x2 - 6

def dfdx2(x1,x2):
   return 2*x2 + x1 - 5

def gradfun(currpoint):
   x1 = currpoint[0]
   x2 = currpoint[1]
   return np.array([dfdx1(x1,x2),dfdx2(x1,x2)])
```



$\lambda = 0.05$ $\lambda = 0.2$ $\lambda = 0.42$

```
def gradient_descent(gradient, start, learn_rate, n_iter):
    currpoint = start
    for _ in range(n_iter):
        diff = -learn_rate * gradient(currpoint)
        currpoint += diff
    return currpoint
```


15

Least square gradient descent

Linear regression

Cost (loss) function $f(\mathbf{\Theta}) = \frac{1}{2n}\sum_i\left((\theta_0 + \theta_1 x_i) - y_i\right)^2$

Derivatives $\frac{\partial f}{\partial \theta_1} = \frac{1}{n}\sum_i\left((\theta_0 + \theta_1 x_i) - y_i\right) x_i$

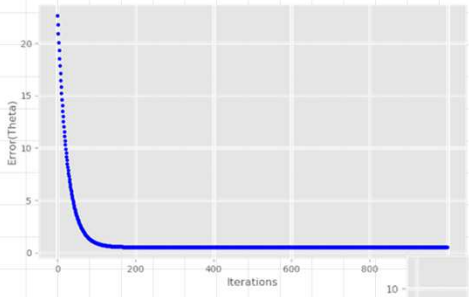
$\frac{\partial f}{\partial \theta_0} = \frac{1}{n}\sum_i\left((\theta_0 + \theta_1 x_i) - y_i\right)$ $\boxed{*\,1}$

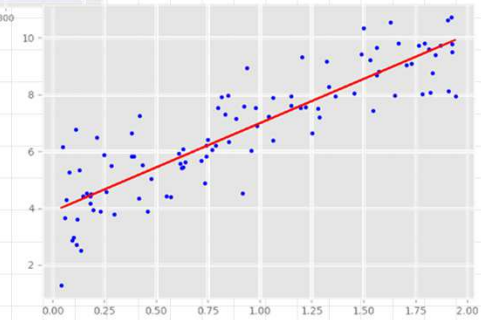
Moving in the opposite direction of the gradient reduces the error (needs to have X as a couple $(1, x_i)$).

```
def gradient_descent(X,y,theta,learning_rate=0.01,iterations=100):
    m = len(y)
    cost_history = np.zeros(iterations)
    theta_history = np.zeros((iterations,2))
    for it in range(iterations):
        prediction = np.dot(X,theta)
        theta = theta - (1/m)*learning_rate*( X.T.dot((prediction - y)))
        theta_history[it,:] =theta.T
        cost_history[it]  = cal_cost(theta,X,y)
    return theta, cost_history, theta_history
```


16

Least square gradient descent





17

Gaussian regression

Gaussian function: $g(x; A, \sigma, \mu) = \frac{A}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$

Derivatives w.r.t. x: $\frac{\partial g(x;A,\sigma,\mu)}{\partial x} = -A \frac{(x-\mu)}{\sigma^3\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$

Derivatives w.r.t. parameters:

- $\frac{\partial g(x;A,\sigma,\mu)}{\partial A} = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
- $\frac{\partial g(x;A,\sigma,\mu)}{\partial \mu} = A \frac{x-\mu}{\sigma^2} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
- $\frac{\partial g(x;A,\sigma,\mu)}{\partial \sigma} = A \frac{(x-\mu)^2}{\sigma^3} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} - \frac{A}{\sigma^2\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$

Simplifying the notation a bit …

18

Gaussian regression 2

Gaussian function: $g(x; A, \sigma, \mu) = Ae^{-\frac{(x-\mu)^2}{2\sigma^2}}$

No σ in the amplitude coefficient

Derivatives w.r.t. parameters:

- $\frac{\partial g(x;A,\sigma,\mu)}{\partial A} = e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
- $\frac{\partial g(x;A,\sigma,\mu)}{\partial \mu} = A\frac{x-\mu}{\sigma^2}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
- $\frac{\partial g(x;A,\sigma,\mu)}{\partial \sigma} = A\frac{(x-\mu)^2}{\sigma^3}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$

Least squares

The error is computed as $f(x_0, y_0) = (g(x_0; A, \mu, \sigma) - y_0)^2$ then averaged over all datapoints.

Gradient descent acts on the parameters of the Gaussian function in order to minimize the error function. The gradients of the error function, to include in the algorithm, are:

Derivatives w.r.t. parameters:

- $\frac{\partial (g(x;A,\mu,\sigma)-y)^2}{\partial A} = 2\left(Ae^{-\frac{(x-\mu)^2}{2\sigma^2}} - y\right)e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
- $\frac{\partial (g(x;A,\mu,\sigma)-y)^2}{\partial \mu} = 2\left(Ae^{-\frac{(x-\mu)^2}{2\sigma^2}} - y\right)A\frac{x-\mu}{\sigma^2}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
- $\frac{\partial (g(x;A,\mu,\sigma)-y)^2}{\partial \sigma} = 2\left(Ae^{-\frac{(x-\mu)^2}{2\sigma^2}} - y\right)A\frac{(x-\mu)^2}{\sigma^3}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$

Least square gradient descent

```
def gradient_descent(x, y, start, lmbda, niter):
    currsol = start
    diff = np.zeros(3)
    for it in range(niter):
       ampl  = currsol[0]
       mu    = currsol[1]
       sigma = currsol[2]
       diff[0] = -lmbda * np.mean( dcostda(x, y, ampl, mu, sigma) )
       diff[1] = -lmbda * np.mean( dcostdloc(x, y, ampl, mu, sigma) )
       diff[2] = -lmbda * np.mean( dcostdscale(x, y, ampl, mu, sigma) )
       currsol += diff
       if it % 1000 == 0:
          print("[{0}] {1}".format(it,currsol))
    return currsol
```


Scipy: curve_fit

SciPy provides the curve_fit() function for curve fitting via nonlinear least squares.

The function takes x and y data as arguments, besides the name of the mapping function to use.

curve_fit() returns the optimal values for the mapping function, i.e., the coefficient values. It also returns a covariance matrix for the estimated parameters.

```
# fit curve
popt, _ = curve_fit(objective, x_values, y_values)
```

Once fit, we can use the optimal parameters and the mapping function objective() to calculate the output for any arbitrary input.

```
# ------------------------------------ scipy
best_vals, covar = curve_fit(gaussian, x, y, p0=init_vals)
print('CF best_vals: {}'.format(best_vals))
yfit = gaussian(x,best_vals[0],best_vals[1],best_vals[2])
```

Parameters setting: grid search

Algorithms are very sensitive to their control parameters (aka hyperparameters).

Optimizing parameter setting is an active area of research, many method, from simple to very complicated.

The simplest one: grid search: define an interval for each parameter and test all combinations (integer parameters).

Example: ARIMA, parameters p,d,q, intervals: $p \in [0,3], d \in [0,2],\ q \in [0,2]$

Grid search:

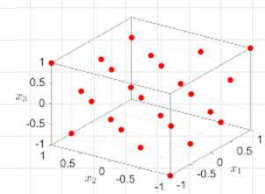
```
for (p=0;p<=3;p++)
   for(d=0;d<=2;d++)
      for(q=0;q<=2;q++)
      {  performance = ARIMA(p,d,q);
         if (performance > best_performance) … etc
```

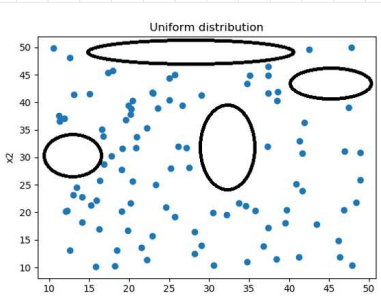

Hammersley sampling

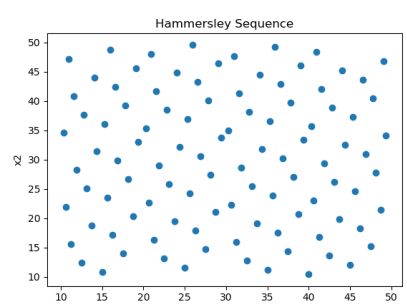
For continuous parameters, it is possible to randomly generate values inside the interval

This could lead to uneven sampling in multidimensional spaces

A possible solution is Hammersley sampling







Hammersley sampling

Hammersley is a representative of methods called "*low discrepancy methods*", nontrivial mathematics involved. Other posssibilities include Sobol and Halton sequences (available in Scipy).

They generate a sequence of n-dimensional points, each of which could correspond to a parameter setting.

Example of output:

```
FINAL: points:
    0      0.00000000      5.00000000      0.00000000      1.00000000      0.00000000
    1      0.33333333      7.85714286      1.87500000      1.47619048      3.00000000
    2      0.66666667      6.42857143      3.75000000      1.95238095      6.00000000
    3      1.00000000      9.28571429      0.62500000      2.42857143      9.00000000
    4      0.00000000      5.71428571      2.50000000      2.90476190     12.00000000
    5      0.33333333      8.57142857      4.37500000      1.09523810     15.00000000
    6      0.66666667      7.14285714      1.25000000      1.57142857     18.00000000
    7      1.00000000     10.00000000      3.12500000      2.04761905      0.42857143
    8      0.00000000      5.35714286      5.00000000      2.52380952      3.42857143
    9      0.33333333      8.21428571      0.20833333      3.00000000      6.42857143
   10      0.66666667      6.78571429      2.08333333      1.19047619      9.42857143
```


Nelder-Mead simplex

https://en.wikipedia.org/wiki/Nelder-Mead_method

Derivative-free methods, several of them. Nelder-Mead simplex is one, essentially a local search.

The Simplex (Nelder-Mead) method:

- Simple to program
- Does not require derivative data
- No general theoretical proof that it works, (but lots of empirical evidence)

Python code on https://github.com/fchollet/nelder-mead

(available on scipy: https://docs.scipy.org/doc/scipy/reference/optimize.minimize-neldermead.html#optimize-minimize-neldermead)

Downhill Simplex (Nelder-Mead)

Keep track of *n*+1 points in *n* dimensions

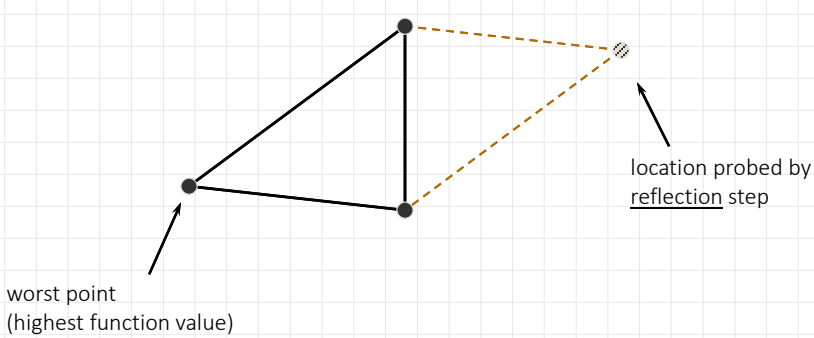
- Vertices of a *simplex* (triangle in 2D tetrahedron in 3D, etc.)

At each iteration: simplex can move, expand, or contract

- Sometimes known as *amoeba method*: simplex "oozes" along the function

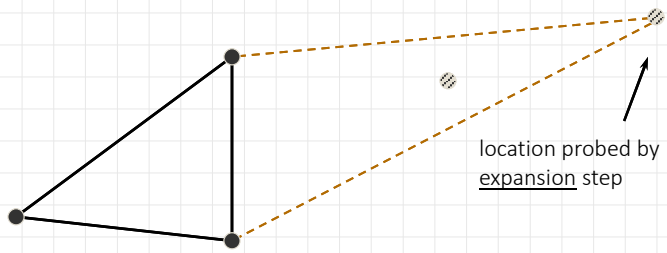
Downhill Simplex (Nelder-Mead)

Basic operation: reflection



Downhill Simplex (Nelder-Mead)

If reflection resulted in best (lowest) value so far,
try an expansion

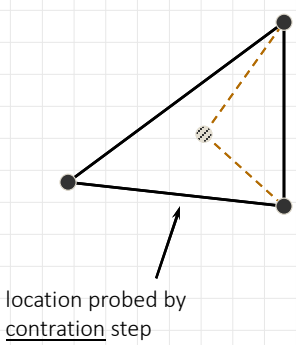


Else, if reflection helped at all, keep it

29

Downhill Simplex (Nelder-Mead)

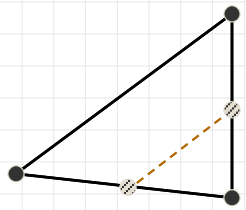
If reflection didn't help (reflected point still worst) try a
contraction



30

Downhill Simplex (Nelder-Mead)

If all else fails shrink the simplex around the *best* point



Downhill Simplex (Nelder-Mead)

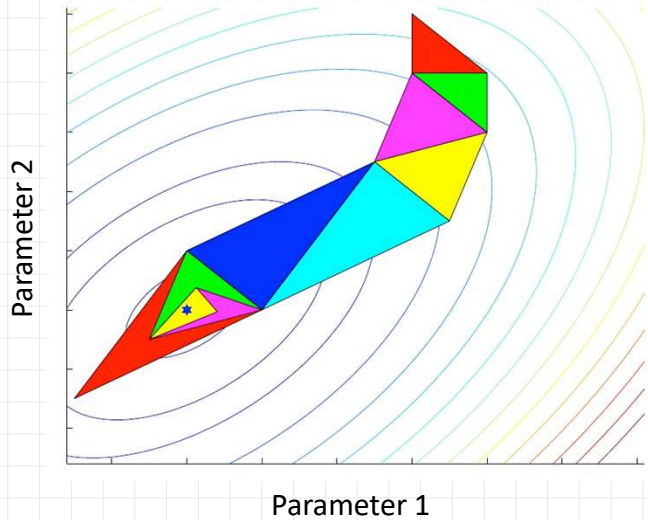
Method fairly efficient at each iteration
(typically 1-2 function evaluations per iteration)

Can take *lots* of iterations

Somewhat flakey – sometimes needs *restart* after simplex collapses on itself, etc.

Benefits: simple to implement, doesn't need derivative, doesn't care about function smoothness, etc.

Nelder-Mead, esempio



33

Nelder-Mead pseudocode

(*from https://en.wikipedia.org/wiki/Nelder-Mead_method*)
We are trying to minimize the function f(x), where $x \in R^n$. Our current test points are $x_1, \ldots, x_{n+1}$

1. Order according to the values at the vertices, $f(x_1) \le f(x_2) \le \ldots \le f(x_{n+1})$.
Check whether method should stop.

2. Calculate x_o, the centroid of all points except x_{n+1}.

3. Reflection
Compute reflected point $x_r = x_o + \alpha(x_o - x_{n+1})$ with $\alpha > 0$.
If the reflected point is better than the second worst, but not better than the best, i.e. $f(x_1) \le f(x_r) < f(x_n)$, then obtain a new simplex by replacing the worst point x_{n+1} with the reflected point x_r, and go to step 1.

4. Expansion
If the reflected point is the best point so far, $f(x_r) < f(x_1)$, then compute the expanded point $x_e = x_o + \gamma (x_r - x_o)$ with $\gamma > 1$.
If the expanded point is better than the reflected point $f(x_e) < f(x_r)$, then obtain a new simplex by replacing the worst point x_{n+1} with the expanded point x_e and go to step 1;
else obtain a new simplex by replacing the worst point x_{n+1} with the reflected point x_r and go to step 1.

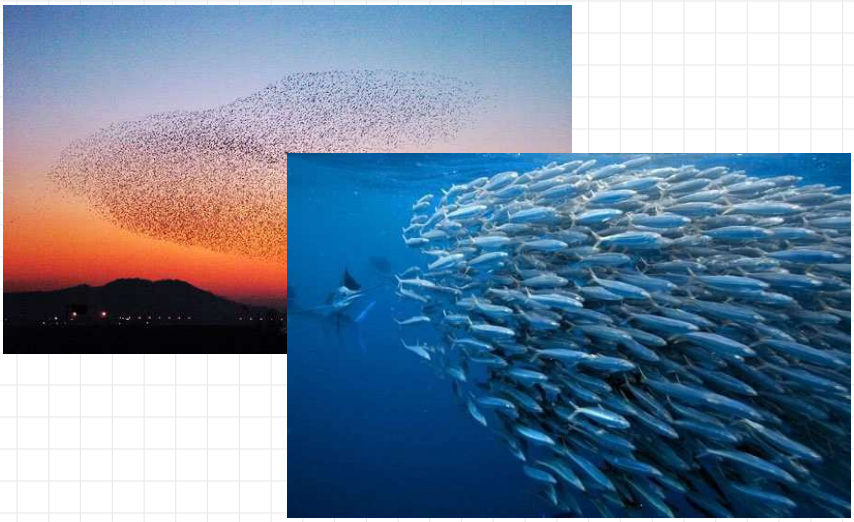
5. Contraction
Here it is certain that $f(x_r) \ge f(x_n)$. (Note that x_n is second or "next" to highest.)
Compute contracted point $x_c = x_o + \rho(x_{n+1} - x_o)$ with $0 < \rho \le 0.5$.
If the contracted point is better than the worst point, i.e. $f(x_c) < f(x_{n+1})$, then obtain a new simplex by replacing the worst point x_{n+1} with the contracted point x_c and go to step 1;

6. Shrink
Replace all points except the best (x_1) with $x_i = x_1 + \sigma (x_i - x_1)$ and go to step 1.

Note: α, γ, ρ and σ are respectively the reflection, expansion, contraction and shrink coefficients. Standard values are $\alpha = 1$, $\gamma = 2$, $\rho = 1/2$ and $\sigma = 1/2$

34

Swarms



Particle Swarm Optimization (PSO)

- PSO is an optimization method inspired by the simulation of behaviors of animals that move in flocks. It uses social interaction as a metaphor to design a problem solving algorithm.
- Proposed in 1995 by James Kennedy (social psychologist) and Russell Eberhart (electrical engineer).
- PSO algorithms are especially useful for parameter optimization in continuous, multi-dimensional search spaces.
- PSO uses agents (particles) which are members of a swarm that moves in the search space looking for an optimal solution.
- Each particle is considered as a point in an *n*-dimensional space that regulates its "flight" according to its flight experience and that of the other particles.

Particle Swarm Optimization

The “swarm” is made by particles in a multidimensional space, each has a position and a speed.

The particles fly in this hyperspace (i.e., the search space) and can do two things:

- memorize their best position (best solution found)
- get to know the global best, and/or the best in their vicinity.

The members of a swarm communicate good positions to each other, and accordingly change their speed, and subsequently positions.

Particle Swarm Optimization

How much influence a particular point has on other particles is determined by its 'fitness‘: a measure assigned to a potential solution, which captures how good it is compared to all other solution points.

Metaphors:

- evolution with the idea of 'survival of the fittest’,
- social behavior through a 'follow the local leader' effect and emergent pattern formation.

Particle Swarm Optimization

Each particle keeps track of its coordinates in the problem space which are associated with the best fitness it has achieved so far. This value is called *pbest*.

Another "best" value is obtained so far by any particle in the neighbors of the particle. This location is called *lbest*.

The best value obtained by the whole population is a global best and is called *gbest*.

Particle Swarm Optimization

PSO search consists of iteratively changing the velocity of (accelerating) each particle toward its *pbest* and *gbest* (global version of PSO) or *lbest* locations (local version of PSO).

Acceleration is weighted by a random term, with separate random numbers being generated for acceleration toward *pbest* and *gbest* / *lbest*.

The idea, recap

Each particle moves at its own speed and remembers its current best result (*particle best, pbest*) and where it got it.

The particles cooperate. They exchange information about their respective results:

- each particle has an associated neighborhood.
- each particle knows its fitness and that of its neighbors.
- each particle modifies its speed according to its local best, global best current (*gbest*) and the position of its neighbor with better fitness (*lbest*).

Particle Swarm Optimization

Each particle modifies its position on the basis of:

- its current position,
- its own current speed,
- distance between current position and *pbest*,
- distance between current position and *gbest*.
- (possibly) distance between current position and *lbest*.

The variation of the particle position is calculated with the formula:

$$v_{ik+1} = c_0 v_{ik} + c_1 rand_1(\dots) \times (pbest_i - x_{ik}) + c_2 rand_2(\dots) \times (gbest_i - x_{ik}) + \dots$$

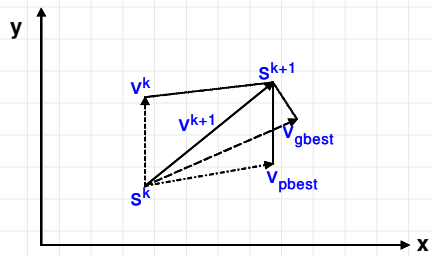
where v_{ik}: velocity of particle i at iteration k,

c_j : weights,

$rand()$: uniform random numbers between 0 and 1,

x_{ik} : current position of particle i at iteration k,

Particle Swarm Optimization



Concept for position update in PSO

x^k : current position.
x^{k+1}: next position.
v^k: current velocity.
v^{k+1}: next velocity.
v_{pbest} : influence of pbest on velocity.
v_{gbest} : influence of gbest (lbest) on velocity

Particle Swarm Optimization

Weights are often dynamically updated with

$$\boldsymbol{c}_0 = \boldsymbol{c}_{max} - [(\boldsymbol{c}_{max} - \boldsymbol{c}_{min}) \times iter]/maxiter$$

where $\boldsymbol{c}_{max}$ = vector of max weights, diversification

$\boldsymbol{c}_{min}$ = vector of min weights, intensification

$maxiter$ = total num if iterations (termination),

$iter$ = current iteration.

PSO Algorithm

1) Initialize the population - locations and velocities
2) Evaluate the fitness of each individual particle (pbest)
3) Keep track of the swarm / neighborhood highest fitness (gbest / lbest)
4) Modify velocities based on pbest and gbest / lbest
5) Update the particles position
6) Terminate if the condition is met
7) Go to Step 2

PSO pseudocode

1. Randomly initialize particle positions *x[i,j]* and velocities *v[i,j]*
2. Initialize the global and local fitness to the worst possible
3. Loop until end_condition
4. evaluate the fitness *f[i]* of each particle
5. update the particle bests *pbest[i,j]* of each particle *i*
6. update the global (neighborhood) best *gbest* (*lbest*)
7. // update of particle velocity and position

for $i = 1$ to number of particles n do

for $j = 1$ to number of dimensions m do

$R1$ = uniform random number

$R2$ = uniform random number

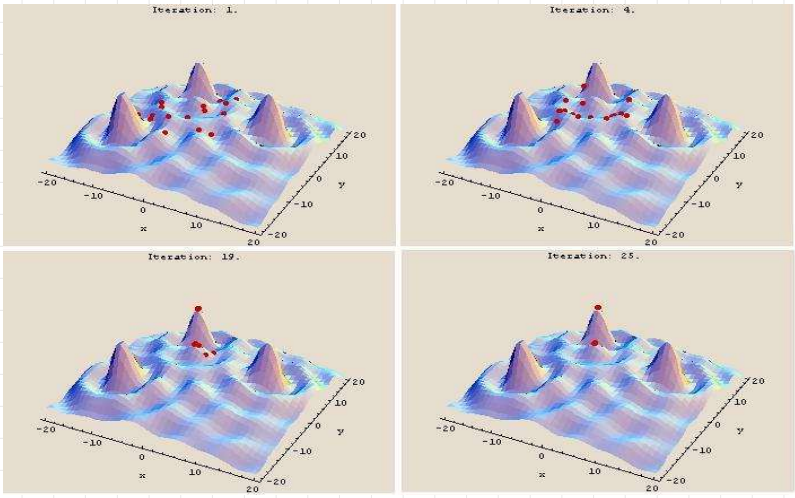
$v[i,j] = w*v[i,j] + C_1*R_1*(pbest[i,j] - x[i,j]) + C_2*R_2*(gbest[j] - x[i,j])$

$x[i,j] = x[i,j] + v[i,j]$

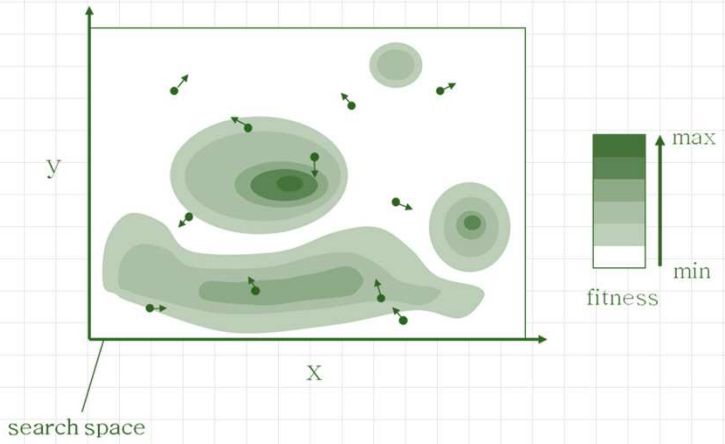
PSO Algorithm

- Number of particles usually between 10 and 50
- C_1 is the importance of personal best value
- C_2 is the importance of global / neighborhood best value
- Often $C_1 + C_2 = 4$ (empirical)
- If velocity is too low, the algorithm is too slow
- If velocity is too high, the algorithm is too unstable

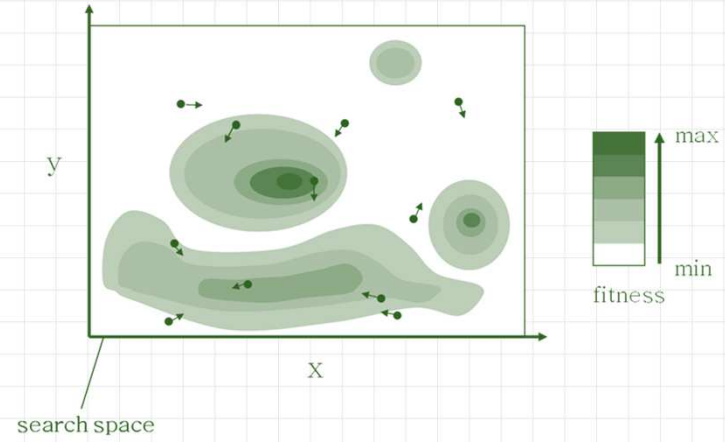
PSO Trace



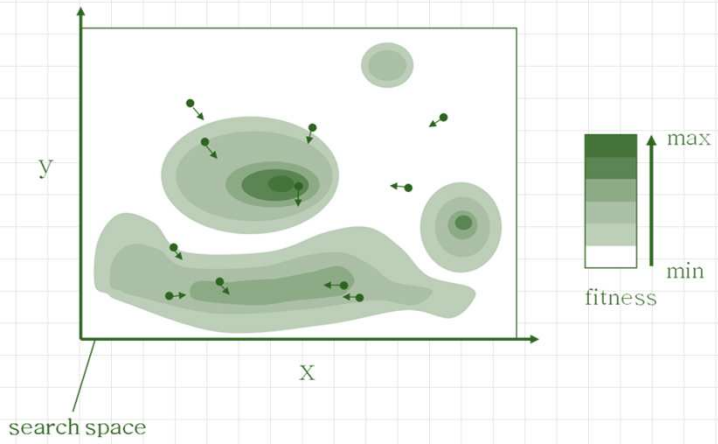
PSO Trace



PSO Trace

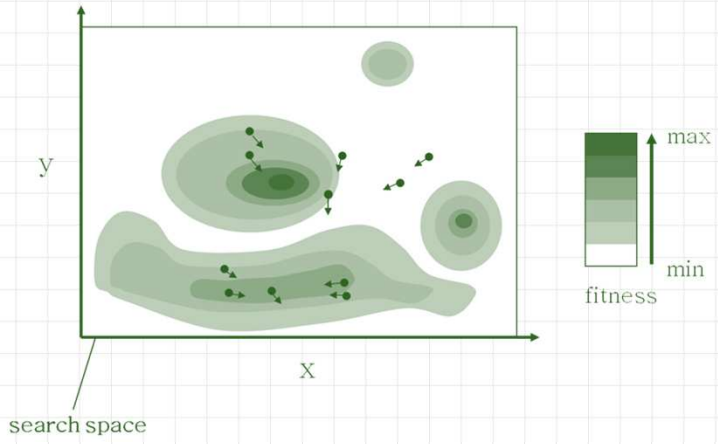


PSO Trace



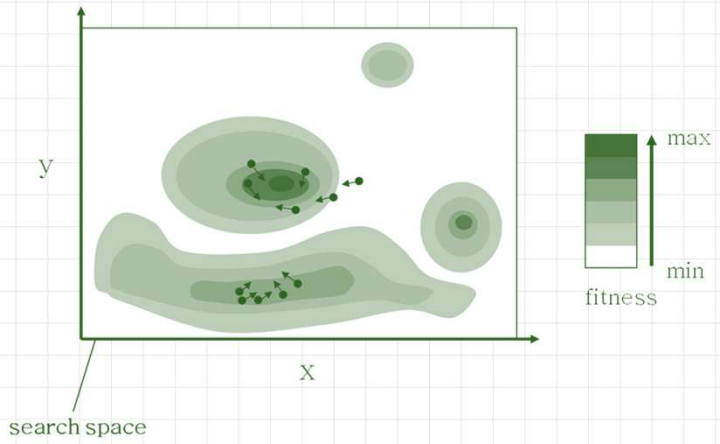
51

PSO Trace



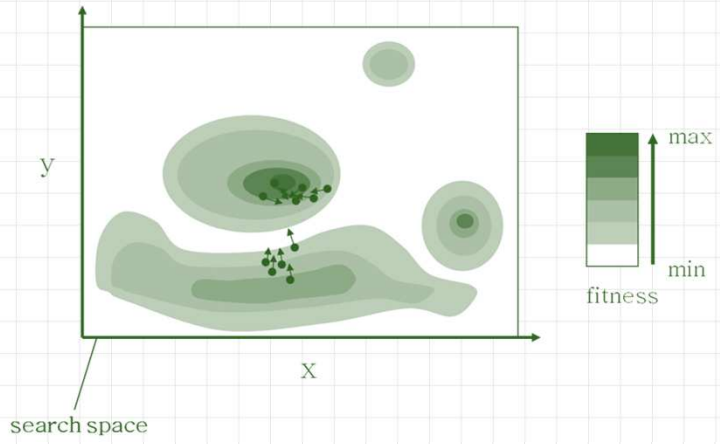
52

PSO Trace



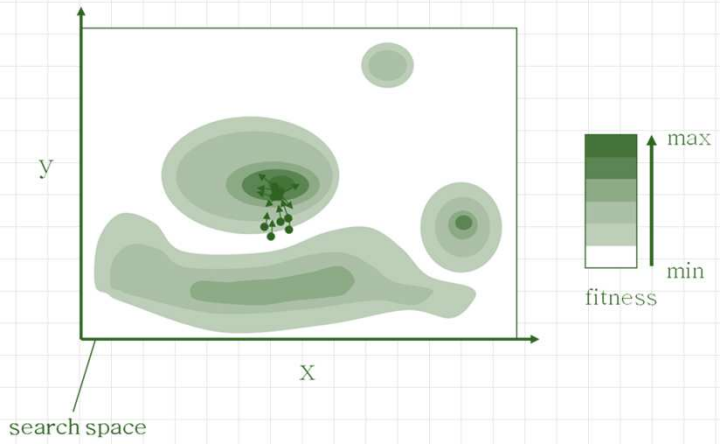
53

PSO Trace



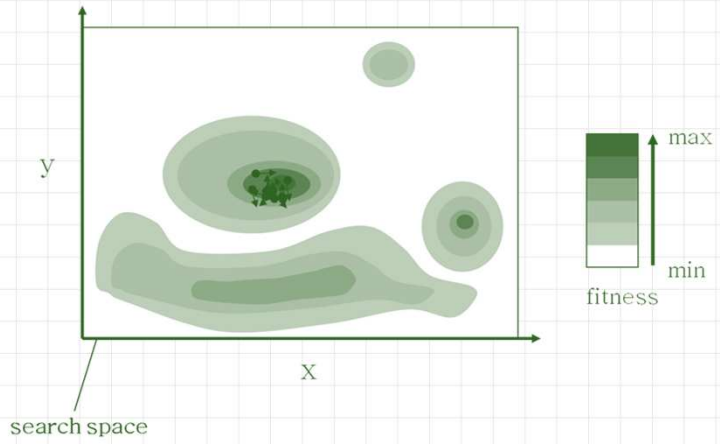
54

PSO Trace



55

PSO Trace



56